SIPE — Smart Income Planner

Project Report

Course: Development of a Functional Business Website System
Student: Briian Njuguna
Email: njugunabriian.dev@gmail.com
Date: April 2026
Deadline: 30 days from commencement

Table of Contents

1. Problem Identification and Planning
2. System Description
3. Target Users
4. System Features
5. Tools and Technologies Used
6. Architecture Overview
7. Database Schema
8. User Interface Design
9. Steps to Run the System
10. Cybersecurity Measures
11. Conclusion

---

1. Problem Identification and Planning

1.1 The Problem

Freelancers, independent contractors, and self-employed professionals in Kenya receive irregular income from multiple clients at unpredictable intervals. Unlike salaried employees who can budget around a fixed monthly paycheck, freelancers routinely face challenges such as:

- Failing to set aside money for taxes and professional expenses
- Spending money that should be saved or invested
- Having no clear picture of how much they have earned or spent in any given period
- Losing track of which client payment came from which project

Without a structured system, many freelancers operate reactively — spending what is available rather than allocating income with intention. This leads to financial instability even among high earners.

1.2 The Proposed Solution

SIPE (Smart Income Planner) is a web-based financial management system designed specifically for freelancers. Every payment received through the platform is automatically split into four predefined financial buckets: **S**avings, **I**nvest, **P**ay Yourself, and **E**xpenses. The user sets the percentage allocated to each bucket, and the system handles the distribution automatically every time a payment arrives via Paystack (a widely used African payment gateway).

The name SIPE is both an acronym for the four buckets and a reference to the concept of directing money through a pipeline — income flows in and is channelled purposefully to where it needs to go.

---

2. System Description

SIPE is a cloud-hosted, full-stack web application that integrates with the Paystack payment gateway to automatically record and categorise incoming payments for freelancers. The system provides:

- A **landing page** that explains the product and directs users to sign up
- An **authentication system** (registration and login) powered by Supabase Auth
- A **dashboard** showing bucket balances, monthly income/expenditure summaries, and a chronological transaction history grouped by month
- A **transactions page** for viewing all deposits and expenses with advanced filtering
- A **payment links page** where users can generate Paystack-hosted payment pages to share with clients
- A **settings page** where users configure their S/I/P/E allocation percentages and retrieve their unique Paystack webhook URL

When a client pays through a Paystack payment link, Paystack sends a webhook notification to SIPE. The system verifies the request signature, records the income, and automatically creates four allocation records — one per bucket — based on the user's configured percentages. This entire process happens without any manual input from the user.

---

3. Target Users

Primary users: Freelancers and self-employed professionals in Kenya, including:

- Graphic designers, photographers, and videographers
- Software developers and IT consultants
- Writers, translators, and content creators
- Marketing consultants and social media managers
- Any individual who invoices clients and receives irregular income

Secondary users: Small business owners who issue invoices and want automatic income categorisation without adopting complex accounting software.

Technical profile of users: The system is designed for non-technical users. No coding knowledge is required. The only technical step is pasting a webhook URL into the Paystack dashboard, which is explained clearly within the application.

---

4. System Features

The following features are implemented and functional in the current system:

#	Feature	Description
1	**User Registration & Login**	Secure email/password authentication with session management
2	**Automatic Income Allocation**	Every payment received is automatically split into four S/I/P/E buckets based on user-defined percentages
3	**Payment Link Generation**	Users create Paystack-hosted payment pages that clients can pay through without needing a Paystack account

4	**Paystack Webhook Integration**	Incoming payments are recorded automatically in real time via a cryptographically verified webhook
5	**Manual Expense Recording**	Users log outgoing expenses, assigning each to a bucket (e.g. software subscription charged to Expenses bucket)
6	**Dashboard with Bucket Balances**	Visual overview of current balance, allocated funds, and spending progress for each of the four buckets
7	**Transaction History with Filtering**	Full history of all income deposits and expenses, searchable and filterable by date range, amount, and bucket
8	**Configurable Allocation Percentages**	Users set their preferred S/I/P/E split in Settings (must total 100%)
9	**Monthly Income Summary**	Dashboard groups transactions by month showing total income and spending per period
10	**Idempotent Webhook Processing**	Duplicate payment events from Paystack are safely detected and ignored

---

5. Tools and Technologies Used

5.1 Front-End

Tool	Purpose
React 18	JavaScript UI library for building component-based interfaces
TypeScript	Strongly typed superset of JavaScript for safer, more maintainable code
Vite	Fast front-end build tool and development server
Tailwind CSS	Utility-first CSS framework for responsive, consistent styling
shadcn/ui	Pre-built accessible UI component library (buttons, modals, inputs)
Lucide React	Icon library used throughout the interface
Sonner	Toast notification library for user feedback (success/error messages)
React Router DOM	Client-side routing between pages

5.2 Back-End and Cloud Infrastructure

Tool	Purpose
Supabase	Open-source Firebase alternative providing PostgreSQL database, authentication, and edge function hosting
Supabase Auth	Handles user registration, login, session management, and JWT tokens
Supabase Edge Functions	Serverless functions running on Deno (used for webhook handling and Paystack API calls)

PostgreSQL	Relational database (hosted on Supabase) storing all application data
Row Level Security (RLS)	PostgreSQL security policy ensuring users can only access their own data

5.3 External APIs

Tool	Purpose
Paystack	African payment gateway used to create payment pages and receive payment webhooks
Paystack Payment Pages API	Creates hosted payment pages that clients use to pay
Paystack Webhooks	Real-time event notifications sent to SIPE when a payment succeeds

5.4 Development Tools

Tool	Purpose
Node.js / npm	Package management and build toolchain
Git	Version control
Supabase CLI	Local development, database migrations, and edge function deployment
VS Code	Primary code editor

---

6. Architecture Overview

SIPE follows a modern **JAMstack architecture** — a static front-end that communicates with cloud back-end services via APIs.

```
┌──────────────────────────────────────────────────────────┐
│                        CLIENT                            │
│           React + TypeScript (Vite SPA)                  │
│  Dashboard │ Transactions │ Links │ Settings │ Auth       │
└──────────────────────────┬───────────────────────────────┘
                           │ HTTPS (Supabase JS SDK)
                           ▼
┌──────────────────────────────────────────────────────────┐
│                     SUPABASE CLOUD                        │
│                                                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │  Supabase    │  │  PostgreSQL  │  │    Edge      │   │
│  │    Auth      │  │  Database    │  │  Functions   │   │
│  │  (JWT/email) │  │  (RLS on all │  │  (Deno       │   │
│  │              │  │   tables)    │  │  serverless) │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
```

```
                                        │ HTTPS
                                        ▼
                                ┌──────────────────────┐
                                │   PAYSTACK API       │
                                │  - Payment Pages     │
                                │  - Webhooks          │
                                └──────────────────────┘
```

6.1 Data Flow — Automatic Payment Recording

The following sequence describes what happens from the moment a client makes a payment to when it appears on the user's dashboard:

1. The SIPE user creates a **Payment Link** in the app (which calls the `create-payment-link` edge function, which calls the Paystack API to create a hosted payment page).
2. The user shares the payment link URL with their client.
3. The client opens the link, enters their card details, and completes the payment on Paystack's hosted page.
4. Paystack sends a `charge.success` webhook event to the SIPE user's personalised webhook URL: `{SUPABASE_URL}/functions/v1/paystack-webhook?uid={user_id}`.
5. The `paystack-webhook` edge function: (a) verifies the request signature using HMAC-SHA512, (b) extracts the amount and payment details, (c) creates one **parent income transaction** row, (d) creates four **child allocation rows** — one per bucket — based on the user's saved percentages.
6. The dashboard reloads and displays the new income, bucket balances updated accordingly.

6.2 Edge Functions

Three serverless functions handle all server-side logic:

Function	Trigger	Responsibility
`paystack-webhook`	Paystack `charge.success` event	Verify signature, record income, create bucket allocations
`record-deposit`	Authenticated frontend call	Manually record a deposit and split it into buckets
`create-payment-link`	Authenticated frontend call	Create a Paystack Payment Page via Paystack API, save to database

---

7. Database Schema

The database is hosted on Supabase (PostgreSQL). All tables enforce Row Level Security — no user can read, write, or delete another user's data.

7.1 Tables

`profiles`

Stores basic user information.

Column	Type	Notes

`id`	UUID	Primary key, references `auth.users`
`full_name`	TEXT	User's display name
`email`	TEXT	User's email address
`currency`	TEXT	Default: `KES`
`created_at`	TIMESTAMPTZ	Auto-set on insert

`allocation_settings`

Stores each user's S/I/P/E percentage configuration.

Column	Type	Notes
`id`	UUID	Primary key
`user_id`	UUID	Foreign key → `auth.users`
`savings_pct`	INTEGER	Default: 20
`invest_pct`	INTEGER	Default: 15
`pay_pct`	INTEGER	Default: 50
`expenses_pct`	INTEGER	Default: 15
`updated_at`	TIMESTAMPTZ	Updated on save

> *Note: A database trigger (`handle_new_user`) automatically creates both a `profiles` row and an `allocation_settings` row with default values whenever a new user registers.*

`transactions`

The central table recording all financial activity.

Column	Type	Notes
`id`	UUID	Primary key
`user_id`	UUID	Foreign key → `auth.users`
`type`	TEXT	`'income'` or `'expense'`
`bucket`	TEXT	`'S'`, `'I'`, `'P'`, or `'E'` (null for parent rows)
`amount`	NUMERIC	Amount in KES
`description`	TEXT	Human-readable description
`category`	TEXT	Optional expense category
`source`	TEXT	Payer email (for income rows)
`paystack_ref`	TEXT	Paystack transaction reference (unique)
`payment_link_id`	UUID	Foreign key → `payment_links` (optional)

`parent_id`	UUID	Self-referencing FK for allocation children
`occurred_at`	TIMESTAMPTZ	When the transaction took place

Transaction tree model: Every income deposit creates one *parent* row (`parent_id = null` , no bucket) and four *child* rows (one per bucket, `parent_id` pointing to the parent). This design makes it possible to display total income without double-counting the allocation rows.

`payment_links`

Stores Paystack Payment Pages created by users.

Column	Type	Notes
`id`	UUID	Primary key
`user_id`	UUID	Foreign key → `auth.users`
`name`	TEXT	Link name (shown to clients)
`description`	TEXT	Optional description
`amount`	NUMERIC	Fixed amount in KES
`paystack_slug`	TEXT	Paystack's URL slug
`paystack_url`	TEXT	Full shareable URL
`paystack_id`	INTEGER	Paystack's internal page ID
`created_at`	TIMESTAMPTZ	Auto-set on insert

7.2 Views

`bucket_balances`

A database view that computes the current financial state of each bucket per user.

Column	Description
`user_id`	The user this row belongs to
`bucket`	S, I, P, or E
`allocated`	Total income allocated to this bucket
`spent`	Total expenses recorded against this bucket
`balance`	`allocated - spent`

---

8. User Interface Design

8.1 Design Principles

- **Dark theme** with a glassmorphism aesthetic — semi-transparent cards on a dark gradient background

- **Responsive layout** — all pages use responsive grid classes and adapt between mobile and desktop viewports
- **Minimal cognitive load** — each page has a single primary action; no unnecessary controls
- **Colour-coded buckets** — each bucket (S/I/P/E) has a consistent colour used across badges, progress bars, and labels

8.2 Pages

Route	Page	Description
`/`	Landing Page	Public homepage explaining the product with call-to-action buttons
`/register`	Registration	Email/password sign-up form
`/login`	Login	Email/password login form
`/dashboard`	Dashboard	Bucket balances, monthly summary cards, transaction history grouped by month
`/transactions`	Transactions	Tabbed view of deposits and expenses with search/filter controls
`/links`	Payment Links	List of created payment links with shareable URLs
`/links/:id`	Link Detail	Transactions associated with a specific payment link
`/settings`	Settings	Profile editing, S/I/P/E percentage sliders, webhook URL display

8.3 Forms Implemented

- **Registration form** — name, email, password
- **Login form** — email, password
- **Add Expense form** — amount, bucket, category, description, date
- **Create Payment Link form** — name, description, fixed amount
- **Settings form** — full name, four percentage sliders with live total validator
- **Manual Deposit form** — amount, description, date (for income not via Paystack)

---

9. Steps to Run the System

9.1 Prerequisites

- Node.js 18 or later
- A Supabase account (free tier) at supabase.com
- A Paystack account at paystack.com
- Supabase CLI installed: `npm install -g supabase`

9.2 Local Development Setup

Step 1 — Clone the repository and install dependencies

```
git clone <repository-url>
cd sipe-smart-income-planner
```

```
npm install
```

Step 2 — Create a Supabase project

1. Go to supabase.com and create a new project
2. Note down your Project URL and Anon Key from Project Settings → API

Step 3 — Configure environment variables

Create a file named `.env` in the project root:

```
VITE_SUPABASE_URL=https://your-project-id.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key-here
```

Step 4 — Apply database migrations

```
supabase link --project-ref your-project-id
supabase db push
```

Step 5 — Configure Paystack secret

```
supabase secrets set PAYSTACK_SECRET_KEY=sk_test_your_key_here
```

Step 6 — Deploy edge functions

```
supabase functions deploy paystack-webhook --no-verify-jwt
supabase functions deploy record-deposit
supabase functions deploy create-payment-link
```

Step 7 — Start the development server

```
npm run dev
```

The application will be available at `http://localhost:8080`.

9.3 Configuring Paystack Webhooks

1. Log in to your Paystack dashboard
2. Navigate to **Settings → Webhooks**
3. Register your account in the SIPE application
4. Go to **Settings** in the SIPE app and copy your personalised webhook URL
5. Paste it into the Paystack webhook field and save

From this point, every successful Paystack payment will automatically appear in your SIPE dashboard.

9.4 Production Build

```
npm run build
```

The output in the `dist/` folder can be deployed to any static hosting provider (Vercel, Netlify, GitHub Pages, etc.).

10. Cybersecurity Measures

Security was a core consideration throughout development, not an afterthought:

Measure	Implementation
Webhook signature verification	Every incoming Paystack webhook is verified using HMAC-SHA512 with the Paystack secret key. Any request with an invalid signature is rejected with HTTP 401.
Row Level Security (RLS)	All Supabase tables have RLS policies enforced at the database level. A user's JWT token is validated server-side and they can only query or modify their own rows — even if they manipulate API calls.
JWT-authenticated edge functions	`record-deposit` and `create-payment-link` verify the user's Bearer token before processing any request.
Service Role Key isolation	The Supabase service role key (which bypasses RLS) is only used inside edge functions running in Supabase's secure environment — never exposed to the browser.
Idempotent payment processing	Before recording any payment, the webhook handler checks whether the Paystack reference already exists in the database. Duplicate events are silently acknowledged and discarded, preventing double-recording.
Environment variable secrets	All sensitive keys (Paystack secret, Supabase service role) are stored as server-side secrets via the Supabase CLI — never embedded in front-end code or version control.
Input validation	Allocation percentages are validated to sum to exactly 100 before saving. Payment amounts must be positive. Required fields are enforced both on the frontend and inside edge functions.

---

11. Conclusion

SIPE addresses a real, underserved problem for the growing freelancer economy in Kenya. The system successfully demonstrates the integration of modern front-end development, cloud back-end services, third-party payment APIs, and database design into a cohesive, production-ready application.

Summary of Rubric Criteria Met

Criterion	Status
Identifies a real business problem	✅
Describes target users	✅
Lists at least five system features	✅ (ten features implemented)
Web pages built with HTML, CSS, JavaScript	✅ (React/TypeScript compiles to these)
System is easy to use, visually clear, mobile-friendly	✅

Homepage created	✅
Form pages created	✅ (registration, login, add expense, create link, settings)
Server-side functionality developed	✅ (three Supabase Edge Functions)
User registration/login	✅
Data submission	✅
Search or filtering	✅
Front-end connected to back-end	✅
Working database created	✅ (Supabase PostgreSQL)
Save data	✅
Display data	✅
Update data	✅
Delete data	✅
Correct data types and relationships	✅
System description written	✅
Tools used documented	✅
Steps to run the system documented	✅

The system is live, tested against real Paystack payments, and ready for demonstration.

Report prepared for academic submission. All features described herein are implemented and functional.